Bit / Byte / Int / Float
Assembly / x86-64
Y86-64 / Processor Architecture
Storage Device
Cache
Optimization
Linking
Exception
System-level I/O
Virtual Memory
Dynamic Memory Allocation
Network Basis / Programming
Concurrent Programming

Godbolt Reference: https://godbolt.org/z/7f8TdWxKM

- 记号 `Ref. <func>-<Ox> ...` 意为 "请观察 `func` 函数在 `-Ox` 优化级别下的结果".
  - 使用 C++ 是为了方便用 `attribute` 和一些 `constexpr` 语法, 和 C 的结果是一致的.

Bit / Byte / Int / Float

长风长风飘在山海间 / 白鸟白鸟展翅入苍天

- $\mathrm{U2B}, \mathrm{B2U}, \mathrm{T2B}, \mathrm{B2T}, \mathrm{T2U}, \mathrm{U2T}$ 是啥? $\mathrm{x2y}(a)$: 将 a 以 x 表示规则翻译为 **位向量**, 将视作 y 表示规则下的 **数**. (这句话有不说人话的嫌疑, 谢罪.)
  - T: two's complement, 即补码.
  - 在位向量的视角下这些函数都是常函数.

- 正/负溢出 (positive/negative overflow) 是啥? 两个正有符号整型相加溢出到负数, 是 **正溢出**. 反之同理.
  - 将 `signed` 视作对无符号 pattern 的翻译是合理的. 溢出行为无非是对无符号数自然相加 (在 $\mathbb{Z}_{2^w}$ 下), 然后重新翻译.

- 整型算数的隐式转化: `char < short < int < long`, 同宽时 `signed < unsigned`,
  - 提升时: **提升到 operands 的最高等级**, 并且 **至少是** `int`. 确定提升目标后:
    - 若是同位宽提升 (`signed` 到 `unsigned`): pattern 不变. 负数可能变成大正数. **注意循环时的问题**.
    - 若是扩展位宽提升: 总是先符号扩展, 再解释 `signed/unsigned`.
  - 截断时: 直接 **保留低位**.
    - 若截断结果是 `signed`, 截断前后值不相等 (被遗弃位不形如符号扩展), 是 UB.
- `long double` 在 x86-64 下实际存储大小为 $10\mathrm{B}$.
  - 为满足内存对齐要求, 编译器会将它 pad 到 $12\mathrm{B}$ 或者 $16\mathrm{B}$.

- 位移 (shift) 量为负数或者 **不小于** (而非大于) 位长时, 为 UB.
  - 但一般物理效果是: 只取位移量的低若干位作为位移值. 例如 `(int)a >> 37` 可以期待为 `(int)a >> 5`.
  - x86-64 中位移指令 `sal sar shl shr` 的位移量只能来自 `%cl` (与上一点互相印证).

- 有符号乘法的规则到底是啥? `x * y == (int)((unsigned)x * (unsigned)y)` (认为溢出时截断).
  - 将 `signed` 视作对无符号 pattern 的翻译是合理的.
  - 在不考虑溢出时, $\mathrm{T2U}$ 和 $\mathrm{U2T}$ 尊重 $\mathbb{Z}_{2^w}$ 的环运算.
    - More: 在全乘法时 (e.g. `imulq/mulq S`: `R[%rdx]:R[%rax] <- R[%rax] * R[S]`), 符号影响结果, 但也只影响最高位是 0 扩展还是符号扩展.

- `(x << k) == x * pwr_2_k` 对一切整型成立, 但是对有符号数, `/` **向零舍入**, `>>` **向下舍入**.
  - `Ref. idiv128-O2 idiv128_pos-O2`. 这还告诉我们, 已知有符号数非负时, 写成无符号更有益编译器优化.

- IEEE 754 浮点: `float 1+8+23`, `double 1+11+52`. E 的 bias 是 $2^{e-1}-1$. 非规格化, 规格化 (M 忽略 $1.$), `inf`, `nan`.
  - 浮点运算: 假装算出 $x \operatorname{op} y$ 的精确值 z, 然后将 z **向偶数舍入** 为浮点数存储.
  - 注意 **向偶数舍入** 是 **四舍五入** 加上对舍去部分 **恰为** `100...0` 的特判.
  - 加法 **交换但不结合**, 0 是单位, 仅考虑实数值时逆元存在且保序.
  - 乘法 **交换但不结合**, 不对加法分配, 1 是单位, 仅考虑实数值时对非负数乘法保序.
  - 典: $x=y=2^{31}-11$, $z=2^{31}-12$, `double` 下 $x\times y\times z$ 不结合. `Ref. mult_assoc_bomb-Og`.
  - 向整型转化: **向零舍入**. 溢出或 `nan` 时, UB, 但一般是设为 `TMin`.

- Sun, PCC Mac 和互联网传输用 **大端序**.
- **任何时间任何地点任何题目任何难度记住判断字节序是小端还是大端.**

Assembly / x86-64

从此江河入海 / 再无幻想 / 你说听起来 / 真叫人怅惘

- Architecture = ISA: 处理器 "**有什么** (程序员可见状态, 运算等)".
  - 怎么实现? Microarchitecture.
  - 怎么用? 和它没关系 (recall: 第二次月考).
  - 例子:
    - Intel: x86, IA32, Itanium, x86-64.
    - ARM: 被用在几乎所有手机.
    - RISC V: New open-source ISA.

- Machine instructions implement the mechanisms, but the choices are determined by designers. These choices make up the **Application Binary Interface (ABI)**. ~~黄底黑字覆盖加粗, 非常好课件.~~

- CISC 与 RISC 的区别 (部分):
  - RISC 没有条件码, 测试指令会向寄存器存入 0/1 结果;
  - RISC 是 **register-oriented instruction set**: 更多寄存器, 减少基于栈的过程调用;
  - 而 CISC 是 **stack-oriented instruction set**.

- x86-64 **整数 通用** 寄存器: `%r[a-d]x %r[s,d]i %r[s,b]p %r[8..15]` (`%rip` 属于 **特殊寄存器**).
  - ~~词性~~ 位宽变换:
    - `%rN` $\to$ `%rNd` $\to$ `%rNw` $\to$ `%rNb`;
    - `%r[a-d]x` $\to$ `%e[a-d]x` $\to$ `%[a-d]x` $\to$ `%[a-d]h:%[a-d]l` **特殊: 可用** `h` **表示低** 16 **中的高** 8 **位**;
    - 其他 `%rXY` $\to$ `%eXY` $\to$ `%XY` $\to$ `$XYl`;
  - `D(Rb,Ri,S)` 模式中, `Ri` 不能是 `%rsp`, 这个倒是可以理解, 注意 `leaq` 也可能引入这个问题.
  - `cmov` 只能对 `2/4/8` 字节寄存器运算, 不能带字长后缀 (自动推断).

- R-S Latch (锁存器), Edge-triggered Latch, 见课件 ICS09.

- SSE3:
  - XMM Regs: 16 个, 每个 $16\mathrm{B}$, 可以划分为若干个窄数据的 pack.
  - 返回值`%xmm0`, 参数 `%xmm0 %xmm1 ...` **(也是从 0 开始)**.
  - 全是 caller-saved.

- 不太平凡的 condition combination:
  - `l` iff `SF ^ OF`, **这是基础**, 讨论一下就行.
  - `le` iff `(SF ^ OF) | ZF`, 即 `l | ZF`.
  - `g` iff `!(SF ^ OF) & !ZF`, 即 `!le`.
  - `ge` iff `!(SF ^ OF)`, 即 `!l`;
  - `b` iff `CF`, `b` 是 unsigned Below;
  - `a` iff `!CF & !ZF`, 即 `!b & !ZF`.
  - ~~搭 Turing Complete 的时候打开 ICS 课件抄的含金量.~~

- x86-64 过程控制:
  - 整型返回值 `%rax`, 参数 `%rdi %rsi %rdx %rcx %r8 %r9` 然后上栈, 栈 **顶** (最低地址位置) 是 `Arg[7]`.
  - 栈帧结构: `Ref. many_arg_caller-Og many_arg_callee-Og`.

```
|| Saved Regs | Local Vars | Args Construct | Ret Addr for callee ||
... CALLEE FRAME
```

- 结构体参数一般在 caller 构造好, 放在 **caller** 的栈帧, 并由 caller 向 callee 传递 **对应地址**.
- **只有** `%rbx %rbp %r[12..15]` (两个带 `b` 的 + 数字后一半) 是 callee-saved 的, (包括所有 `%xmmN` 在内的) 其他寄存器都是 caller-saved. `%rsp`: **"special form of callee save"**.

- C 类型定义: **右左法则**. 其实看个例子就会了: `int *(*(*p[3])(int*))[5]`, 则
  - `p[3]`: `p` 是一个长度为 3 的数组,
  - `*p[3]`: 元素是指针,
  - `(*p[3])(int*)`: 指向一个参数为 `int*` 的函数,
  - `*(*p[3])(int*)`: 函数返回值是一个指针,
  - `(*(*p[3])(int*))[5]`: 指向一个数组,
  - `*(*(*p[3])(int*))[5]`: 这个数组的元素是指针,
  - `int *(*(*p[3])(int*))[5]`: 指向 `int`. 完结撒花.

- 当低 32 位寄存器名作为目标寄存器时, 高 32 位会被清零, 但低 $n < 32$ 位寄存器名作为目标寄存器时, 其他位不变.
  - 例如, `setX R` 只设置 `R` 的最低字节, 不改变其他字节.
  - 用 `setX %al` 加上 `movzbl %al %eax` 赋值. (`zbl`: Zero-padding Byte to Long word, 或许吧, 反正好记).

- C switch 跳转表: `goto *JTab[x];` $\rightarrow$ `jmp *.L1(,%rdi,8)` (例子).

Y86-64 / Processor Architecture

你亲自画满 / 世界的俯瞰 / 好过听说一万种答案

- Y86-64 基础 & Seq 实现:
  - 状态: x86-64 的 16 个去掉 `%r15`, `ZF OF SF PC Stat DMEM`.
  - 指令: `halt nop cmovXX irmovq rmmovq mrmovq OPq jXX call ret pushq popq`, 共 12 个:
    - 注意哪些有后缀 `q`.
    - 我们总对 `rB` 处理 `D(R)` 的地址偏移, 所以 `mrmovq` 的参数应该是 `mrmovq D(rB), rA`.
    - 注意 `V` 和 `D` 的编码是小端序.
    - `jXX` 和 `callq` 都是绝对地址. c.f. x86-64 在直接跳转/直接调用时都是 PC 相对寻址.
      - Recall: x86-64 的上述 PC 相对寻址计算的都是 **目标地址** $-$ **下一条指令地址**.
      - Recall: 间接跳转/调用形如 `jmp *%rax`, `call *(%rax)`.
    - 条件码判断: `/ le l e ne ge g`.
  - Sequencial 实现:

    - `popq rA` 在 D 阶段是 `valA <- R[%rsp]; valB <- R[%rsp]`, `valA` 用来在 M 寻址, `valB` 用来在 E 计算新 `%rsp`.
    - `cmovXX rA, rB` 通过在 E 阶段 `if !Cond(CC, ifun) rB <- 0xF` 来禁用更新.
    - 只有 `PC` 的更新使用 `PC increment` 元件, 其他的所有寄存器计算都要过 `ALU`.
    - M 阶段后记得计算程序状态 `Stat`. 会用到 `imem_error dmem_error instr_valid icode(==IHALT?)`;

- Y86-64 中 `pushq %rsp` 压入的是 `pushq` 之前的 `%rsp`, `popq %rsp` 后 `%rsp` 被设置为从栈中取出的值.
  - 虽然是 ISA 决定了硬件实现, 这里可以结合硬件实现记忆. 例如对 `pushq`, `%rsp` (旧的) 值在 D 阶段就准备好了.

- Pipeline Delay: t ps, 指一条指令从进入流水线到离开流水线的总时间 (注意时钟周期受最慢组合逻辑延迟控制).
- Pipeline Throughput: n GIPS (Giga Instructions Per Second), 指每秒能够进入 (或离开) 流水线的指令数量. 设时钟周期为 c ps, 则 $n = \frac{1}{c \times 10^{-12}} \times 10^{-9} = \frac{10^3}{c}$.

- 时钟上升沿, Pipeline 基础.
  - ~~第七章, 时序逻辑, 一点没看.~~
  - 若 `i` 是时钟, 形如 `o = i && !!!i` 就是在时钟上升沿的脉冲! (利用了非门延迟.) 锁存器就使用了这种结构.
  - Pipeline 周期为 **最慢** 组合逻辑延迟 + 流水线寄存器延迟. ~~惨痛教训~~
  - 一个 pipeline "周期" 形如:
    - (本句属于上一周期) 组合逻辑稳定 (包括 stall/bubble flags 等);
    - 当前周期开始, 时钟上升沿到来, (上一周期的) 组合逻辑输出填充入流水线寄存器;
    - 流水线寄存器值输入组合逻辑, 组合逻辑电路开始填充;
    - 组合逻辑稳定, 当前周期结束.
  - FDEMW (指书上的标准五级流水线实现, 下同) 的一些细节:
    - E 阶段只有 `valC valA valB`. 没有 `valP`, 因为需要 `valP` 时意味着 `valP` 需要经过复杂运算 (而非单纯 `PC inc`) 得到, 一定来自 `jXX call`, 它们都不需要 `valA/valB`, 所以 "`E_valP`" 被并入 `E_valA`. (`ret` 时也不需要, 要了也没用.)
    - M 阶段只有 `valE valA`, 没有 `valC` 是因为它参与过 ALU 后肯定没用了, `valA` 还在是因为它可能作为内存地址 (`popq ret`) 或者内存写入值 (`rmmovq`).
    - W 阶段只有 `valE valM`. 那不然呢?

- Stalling & Bubbling.
  - 基本逻辑:
    - 组合逻辑计算的 `stall` 和 `bubble` 是根据 **当前周期** 的状态, 决定 **下一周期** 的某个阶段是否需要 stall/bubble.
    - 结合上面 pipeline 基础的内容, `stall` 和 `bubble` 作用于周期开始阶段:

        - `stall`: 拒绝来自上一周期的组合逻辑输出, 保持流水线寄存器所有状态 (不打开锁存器的写口);
        - `bubble`: 将流水线寄存器直接写为 `nop` 的状态 (需要打开锁存器的写口).
        - 可见它俩是互斥的, 你不能既写又不写.
    - 若 X 阶段需要 stall, 说明 **当前** X 阶段的 **流水线寄存器状态都是正确的**, 但需要重新通过组合逻辑.
    - 若 X 阶段需要 bubble, 说明 **当前** X 阶段的 **上一阶段** 完全是 **混乱邪恶**, X 在下一周期必须遗弃上一阶段的输出, 自然也什么都做不了.
        - 注意 `nop` 并非 "混乱邪恶", 所以如果上一阶段在 **当前周期** 已经被 bubble, X 就不必 bubble.
    - 可以看出 F 阶段比较特殊: 它没有 "上一阶段", 自然 **从不 bubble**. 而如果 F 阶段本身是混乱邪恶, D 一定会 bubble, F 是否 stall 其实无所谓. (*在 return hazard 讨论.*)
- **Load/Use Hazard**
    - 为什么存在? Data hazard 的前递能够解决 `dstE == srcX` 的情况, 本质上是因为 D 和 E 刚好是相邻阶段, E 算出结果时 "错误" 的 D 还没走远 (还在组合逻辑里, 没写寄存器), 我们可以支付一点点延迟的代价 (组合逻辑会变长) 转发 E 的结果. 但它解决不了 `dstM == srcX` 的情况, 因为 D 和 M 不相邻, 当 M 算出结果时错误的 D 都已经填充到 E 的流水线寄存器里了, 已经没救了.
    - `load_use_hazard = E_icode in { IMRMOVQ, IPOPQ } && E_dstM in { d_srcA, d_srcB };`
        - 即, E 阶段是一个内存 load, 且 load 到的目标寄存器正在被 D 读取.
    - 控制: F & D stall, E bubble.
    - 浪费: 一个周期.
- **Misprediction**
    - 为什么存在? 分支预测错了呗.
    - `mispred = E_icode == IJXX && !e_Cnd;` (为啥这个 `Cnd` 的 `C` 要大写呢...)
        - Recall: 预测策略是 always-taken, 所以 `!e_Cnd` 就意味着错误预测.
    - 控制: D & E bubble. 注意 F 之前的 "PC 阶段" 是纯组合逻辑, 所以下一周期的 F 能立即获得正确的 PC 值, 可以正常执行; 但当前周期的 F 和 D 是混乱邪恶, 所以需要各自在 D 和 E 被遗弃.
    - 浪费: 两个周期.
- **Return Hazard**
    - 为什么存在? `ret` 时返回地址需要在 M 阶段获取, 而且没法预测.
    - `ret_hazard = IRET in { D_icode, E_icode, M_icode };`
    - 控制: 教材说 "**F stall, D bubble**", 但只有 "D bubble" 也可以. (*向助教确认过没问题, 助教 claim 这样的答案也会接受, 但还是记住前者吧.*)
        - Recall: `ret` 时正确的 PC 在 `ret` 到达 W 阶段时由 `W_valM` 传递 (这也是为什么 `ret` 到达 M 时还认为是 hazard; 反之我们也可以让 PC 从 `m_valM` 传递到 `Sel PC`, 这样 hazard 就会少一个周期, 但问题是关键路径会变长), 传递到 F 阶段中 `Sel PC` 组合逻辑中.
        - F 是否 stall 其实无所谓, 因为不管 F 要不要 stall, 它都会尝试读错误指令, 而且输出都会被 D 立马遗弃.

        - 浪费: 三个周期.
    - 控制组合
        - 某种程度上其实是伪命题: 比如 `jXX (mis) + ??? (in wrong branch)`, 其实 `???` 不管是 `ret`, 是 EOF 还是 `Genshin Start` 都不应该影响流水线控制: 反正 `???` 都是不会执行的指令, 怎么会影响流水线呢?
        - 我们的策略是尽量实现简单: 有 stall 就 stall, 否则有 bubble 就 bubble. 这是正确的, 简单的, 虽然不一定足够高效.

Storage Device

我失去太多太多 / 笑谈如结局尘埃落定的壮志

- **DRAM** (Dynamic RAM):
    - 每 bit 1 晶体管 + 1 电容;
    - 用于主存和帧缓冲区 (frame buffer);
    - 需要 EDC (error detection and correction).
    - **易失性存储**.
- **SRAM** (Static RAM):
    - 每 bit 6 (或 8) 晶体管;
    - $10\times$ 访问速度, $100\times$ 价格;
    - 用于 cache.
    - "**Maybe**" 需要 EDC.
    - **易失性存储**.
- **SDRAM** (Synchronous DRAM):
    - 不同的时钟信号管理方式;
    - 允许行地址重用 (e.g. `RAS CAS CAS CAS` 而不必 `(RAS CAS)*3`).
- **DDR SDRAM** (Double Data-Rate SDRAM):
    - 在时钟上升下降沿都发送 bit;
    - 由预取 buffer 大小区分类型: DDR (2 bits), DDR2 (4 bits), DDR3 (8 bits), DDR4 (16 bits).
    - *Intel Core i7*: 支持 DDR3/DDR4 SDRAM.
- **xROM**:
    - **ROM**: 制造时编程 (真的 read-only);
    - **PROM**: 可以写入一次;
    - **EPROM**: 光 (UV/X-ray) 可擦除;
    - **EEPROM**: 电子可擦除;
    - **Flash mem / 闪存**: 可在块级 (block-level) 擦除.
        - 约 10^5 次擦除后损坏.
    - 用于固件, **固态硬盘**, 磁盘缓存.

- 多 DRAM 时可以广播地址, 在一个单位访存时间内取出多个字节.

- **磁盘参数**
  - 注意都是 10^3 进制.
  - **记录密度** (recording density, bit/in): 平均每英寸磁道容纳的 bit 数;
  - **磁道密度** (track density, $\mathrm{track}/\mathrm{in}$): 平均每英寸宽的磁盘环道 (radial segment) 容纳的磁道数.
  - **区域密度** (areal denisty, $\mathrm{bit}/\mathrm{in}^2$): 上两个的乘积, 反映平均每平方英寸容纳的 bit 数.
  - 层级划分:

$$\text{磁盘} \xrightarrow{\text{含多个}} \text{盘片} \xrightarrow{\text{分上下两个}} \text{盘面} \xrightarrow{\text{按离轴距离分为多个}}$$

$$\text{区域}(\text{圆环}) \xrightarrow{\text{内部含有多个}} \text{磁道}(\text{环线}) \xrightarrow[\text{分为相同数量个}]{\text{同区域的磁道}} \text{扇区}.$$

    一个区域内的磁道所能划分出的扇区数量显然由最内圈磁道决定. 后三项的划分数量并非常数, 所以一般用平均值衡量.
  - **访问过程**:
    - 寻道 (seek, $\sim 3\text{-}9\mathrm{ms}$, **可估计为旋转延迟的两倍**) $\to$ 等待旋转 (rotation,) $\to$ 数据传递 (transfer, $\sim 0.02\mathrm{ms}$).
    - 时间代价: SRAM $\times 40000$, DRAM $\times 2500$ ~~你刚刚不是说它俩只差十倍吗~~.

- 固态硬盘 (SSD):
  - 层级划分: 闪存 $\to$ 块 ($32 \sim 128$ 页) $\to$ 页 ($4 \sim 512\mathrm{KB}$).
  - **以页为单位读写**;
  - **块被擦除后才能写其中的页** ($\sim 10^5$ 次损坏).
  - 更快, 更节能, 更耐造, 约 $\times 4$ 单位成本 (2019 数据, 相对价格正在下降).

Cache

走吧 / 就算我们无法让大雨停下 / 还有我 / 陪你在雨里放肆奔跑啊

- 局部性:
  - **时间局部性**: "用过的很快再用." 需要考虑: 指令 (循环中), 寄存器, 数据内存值;
  - **空间局部性**: "用过的附近也会用." 需要考虑: 指令 (没有大跳转时), 数据内存区域.

- Cache Miss:
  - **冷不命中**: 第一次 ref 某个块 (cache 才启动时);
  - **容量不命中**: 没有任何空位了;
  - **冲突不命中**: 有空位但 cache 策略不让用.

- Memory Hierarchy:

```
 L0  | L1 L2 L3 |  L4  |  L5  |    L6
Regs |   SRAM   | DRAM | disk | e.g. web
```

- Cache 组织结构:
  - **熟记喵!** $S = 2^s$ 个组, 每组 $E = 2^e$ 个缓存行 (cache line), 一个缓存行包含 valid bit, (dirty bit,) tag 和一个含 $B = 2^b$ 字节的缓存块 (cache block), cache size 定义为 $C = S \times E \times B$ (**不计入行的附加信息**).
  - 读写流程: *见虚拟内存部分*.

- 写回 (w-back) / 写穿透 (w-through) / 写分配 (w-allocate) / 写不分配
  - **写穿透**: 写入 cache 时立即写到内存;
  - **写回**: 推迟内存写入, 直到当前行被驱逐 (**需要维护 dirty bit**);
  - **写分配**: 写时不命中, 取入 cache 然后写在 cache line 上;
  - **写不分配**: 写时不命中, 直接写内存.
  - 搭配: 写穿透+写不分配; 写回+写分配. (明显更符合设计逻辑.)
  - ~~已经不认识 "写" 字子.~~

- **Intel Core i7 缓存层级** ~~byd 这玩意儿还真在课件里~~

$$\text{main} \to \underbrace{\underset{\text{8MB,40}\sim\text{75cyc}}{\text{L3 unified}}}_{E=16} \to \overbrace{\underbrace{\begin{cases} \text{Core 0} \to \underset{\text{256KB,10cyc}}{\text{L2 unified}} \to \begin{cases} \underset{\text{32KB,4cyc}}{\text{L1 d-cache}} \to \text{Regs} \\ \underset{\text{32KB,4cyc}}{\text{L1 i-cache}} \end{cases} \\ \text{Core 1} \\ \vdots \end{cases}}_{E=8}}^{B=64}$$

- Cache 性能指标:
  - **Miss Rate**: $3 \sim 10\%$ for L1, $< 1\%$ for L2 (依赖于容量);
  - **Hit time**: 命中时的访存时间 (数据见上面黄字);
  - **Miss Penalty**: 不命中时的额外时间代价. 对主存 $50 \sim 200\text{cyc}$.
  - 测试时需要缓存暖身 (warm-up).
  - e.g. 在 Core i7 的内存山上, 对 8B 数据, 同一个工作集大小, stride $s = 1..7$ 时不命中率 $s/8$ (数据见上), 吞吐率近线性递减, 但 $s \geq 8$ 次次不命中, 吞吐率基本不变.
  - ~~每度回忆都因想你, 每次访主存都因想你, 每度回忆都因没命中你, 每次访主存都因没命中你.~~

Optimization

不必如四季伟大 / 天山不曾规定谁像她 / 随心而去吧

- ~~**时↑间↓复↑杂↓度↑和↓运↑行↓时↑间↓无↑关!**~~
- btw, 这一段是我回课, 所以虽然助教 claim 不会怎么考, 我还是会补充许多有意思的例子, 你可以在这章看点例子休息一下啦.

- (课件里的) 典型优化手段:
  - 代码移动 **Code Motion**: "循环不变表达式外提."

        - `Ref. rowSum-Og rowSum-O2`: 第一维数组寻址被外提.
    - 强度消减 **Reduction in Strength**: "简单运算替代复杂运算."
        - `Ref. rowSum-Og rowSum-O2`: 第二维数组寻址被变为指针累加移动.
    - 共享子表达式复用 **Share Common Subexpressions**: "复用中间计算结果."
        - `Ref. calc_u-Og calc_u-O2`: 编译器甚至会因式分解!
        - (*可跳过*) `Ref. calc_i-Og calc_i-O2`: 无法优化, 因为因式分解后的计算可能引入溢出问题 (联系下面的 opt. block).
- *其他典型优化手段 (*可跳过*):
    - 死码消除 **Dead-code Elimination**: ~~听上去不太文明~~ "无作用代码直接删掉."
        - `Ref. squareNonnegative-Og squareNonnegative-O0` (`-O0` 只在这里用了一次所以没开 compiler, 你可以自己试试): 编译器有权认为 **程序从不引入 UB**, 进而推断 **发生 UB 才能到达的代码段都是死码**, 直接删!
    - 常量折叠 **Constant Folding**: "编译时计算好所有固定的常量值, 不生成对应计算指令而是直接填入结果."
        - `Ref. fibo-Og fibo-O2`: 编译器甚至会花 16s 卖力地硬算递归 Fibonacci 数列.
    - 归纳变量 **Induction Variable**: "识别循环中线性变化的变量." 用以启发其他的优化.

- Optimization Blockers
    - **编译器有保守性**.
    - 编译器将过程调用视为黑盒.
        - `Ref. toLower-O2 countDigit-O2`: 即使强制内联也不一定能让编译器发现更多性质, 尤其是关于内存区域的不变性.
            - 当然这也是因为 C/C++ 丑陋的指针机制.
            - 也可能和多线程行为有关: 编译器觉得你每次读内存都有深意.
        - 某简答题答案: "1) 行为可能依赖全局变量. 2) 可能修改全局变量." 评价为糖: 静态变量 / pointing-to 数据等一堆东西可以 hack 这个答案.
    - Memory Aliasing 内存别名:
        - 人话: pointing-to 内存区域交叠时可能有 (程序员想都没想过的) 特殊行为, 编译器不得不保持原样.
        - `Ref. i64cpy-O2 i64cpy_restr-O2`: 通过 `__restrict__` 指定内存区域被唯一命名, 让编译器大彻大悟.
    - (*可跳过*) 优化的一大阻碍是程序员智慧 (存在性存疑) 与编译器智慧无法对齐.
        - 用 C++ Attribute 教会编译器! `[[noreturn]] [[assume(...)]] [[unlikely]] ...`
        - Modern C++ 正在做这方面的改进, 例如把库函数非常常见的 `(begin_iterator, end_iterator)` 范式换为 `range` 范式. 减少 "在程序员脑子里 (程序员: `begin` 和 `end` 是同一个容器的区域且 `begin<=end`) 但不在代码里 (编译器: 我咋知道?)" 的性质.

- 现代 CPU:
    - 超标量处理器 / 指令多发射: 一次 fetch 并 issue 多条 (互不影响) 的指令到计算阶段;
    - 多 functional units: 存在多个能够处理同一功能的计算单元;
    - pipelined functional units: 计算单元可流水化填充.

        - latency bound (关联 pipeline delay): 一条计算指令从抵达到算完所需的时间;
        - throughput bound (关联 pipeline throughput): 充分流水化时平均每条指令需要的时间.
    - 可见 **连续且无数据依赖的指令** 更能充分发挥 CPU 性能.

- 循环展开与重结合优化:
    - 理论分析技巧的技巧是, **假设处理器有 oracle**: 充分多发射, 充分利用多 FP, 充分流水化 FP.
    - 记号 `Unroll AxB`: A 路循环展开, 用 B 个变量记录各路结果. $B \mid A$.
    - **Haswell CPU** 参数:
        - `2 load | 1 store | 4 int | 2 fp-mult | 1 fp-add | 1 fp-div`
        - 可流水化的 units (括号内是 latency bound; throughput bound 都是 1): `load(4) store(3) fp-mult(5) fp-add(3)`. **除法都不可流水化**.
        - ~~应该不用记, 但 ICS 赐予我了被害妄想症.~~
    - 例子: 分析 Haswell 上计算 `double` 数组元素乘积的最佳展开策略.
        - 在一次循环内部尽可能填充满所有 `fp-mult` 的 pipeline, 可以填入 $2 \times 5 = 10$ 条乘法指令, 要求它们互不影响. 所以 10×10 unrolling 是很棒的选择. 此外, 刚好有两个 `load`, 所以它可以源源不断地加载乘法 operands, 不会成为瓶颈.
        - ~~虽然这和课件上的实验结果 perfectly match, 但感觉过于理想, 主要还是巧合.~~

Linking

把大山大海 / 变成我另一个怀抱 / 每一只白鸟 / 都可以落脚

- GCC "编译" 全过程: $\mathrm{.c} \xrightarrow{\text{-E}} \mathrm{.i} \xrightarrow{\text{-S}} \mathrm{.s} \xrightarrow{\text{-c}} \mathrm{.o} \xrightarrow{\text{-o}} \mathrm{out}$.
    - 分别由 preprocessor cpp, compiler $\mathrm{cc1}$, assembler gas 和 linker ld 处理.

- **ELF** (Executable and Linkable Format):
    - 可重定位目标文件 ($\mathrm{.o}$), 可执行文件 ($\mathrm{a.out}$), 共享目标文件 ($\mathrm{.so}$) 的同一格式.
    - 从文件头起, 含有如下内容:
        - **ELF header**: 字大小, 字节序, 文件类型 (例如是上述三种的具体哪一个), 目标机器类型, ...
        - **Segment header table** (对可执行文件必要): 页大小, 段的虚拟地址 (例如指定代码段从虚拟地址的 `0x400000` 开始加载), 段大小.
        - **.text section**: 指令代码.
        - **.rodata section**: 只读数据, 包括跳转表, 字符串常量等.
        - **.data section**: 已初始化的全局变量
        - **.bss section**: 未初始化或初始化为零的全局变量和静态变量.
            - **COMMON**: 事实上 **未初始化** 的 **全局变量** 会存在这个伪节.
        - **.symtab section**: 符号表.
        - **.rel.text section**: `.text` 段中的重定位信息 (哪里需要重定位, 怎样重定位).
        - **.rel.data section**: `.data` 段中的重定位信息.
        - **.debug section**: `-g` 时启用, 给出调试时的符号信息.

        - **Section header table** (对可重定位文件必要): 记录每一节的偏移和大小.

- 符号分类:
    - **全局符号**: 在模块内定义, 可被其他模块引用的符号.
        - 所有未被 `static` 修饰的函数和全局变量.
    - **外部符号**: 在模块内引用但在其他模块定义的符号.
    - **局部符号**: linker 不可见.

- 静态链接过程:
    - **符号解析**: "linker 将每个符号引用唯一地对应到一个符号定义."
        - 在目标文件中生成符号表 (symbol table), 表项记录了符号的名字, 大小和位置, (隐式) 记录了符号强弱.
            - **不会生成函数声明的表项**.
        - **强符号**: 函数, 初始化的全局变量.
        - **弱符号**: 未初始化的全局变量, 由 `extern` 修饰的全局变量. (注意函数声明没有符号.)
        - 原则:
            - **不允许强符号重名**.
            - 当一个强符号和若干弱符号重名, 选择强符号.
            - 只有若干弱符号重名, 随便选一个.
            - ~~看来 linker 不太喜欢报 error.~~
        - `extern void noArgs();` 和 `extern void noArgs(void);` 不等价: 编译器觉得前者的意思是 "我不告诉你 `noArgs` 的参数类型".
    - **重定位**: "合并代码和数据段, 为符号引用填入正确地址."
        - 节与符号定义的重定位:
            - 合并同名节.
            - 为节和定义的符号分配运行时内存地址.
        - 符号引用的重定位: **根据重定位表项, 为每个符号引用填入正确内存地址**.
            - 表项信息: `r_offset, r_info (encoded R_SYM and R_TYPE), r_addend`;
            - 基础类型: `R_X86_64_PC32` (32 位 PC 相对地址), `R_X86_64_32(S)` (32 位绝对地址).
            - 处理过程:
                - `refptr = s + r.offset`, 其中 `s` 是节起始地址, `offset` 是节偏移, `refptr` 就定位了当前目标文件中需要填入内存地址的位置 (这个位置接下来 $4\mathrm{B}$ 一般留空为 `00 00 00 00`, 所以如果直接 `objdump` 会出现一些 "不明所以" 的跳转, 其实只是占位而已).
                - 对 `R_X86_64_PC32`, 计算 `refaddr = ADDR(s) + r.offset`, 即这个目标位置的运行时内存位置; 否则对 `R_X86_64_32` 可以认为 `refaddr = 0` (不参与计算, 这里只为统一后续步骤).
                - 填入 `*refptr = (unsigned)(ADDR(r.symbol) + r.addend - refaddr)`. 其中 `ADDR(r.symbol)` 即真正想要引用的目标的运行时地址.

        - *经验上*, `addend` 会配合 PC 相对寻址使用. 回忆在 PC 相对寻址时, 我们需要计算目标地址和 **下一条指令起始地址** 的差值, 对 32 位寻址, `refaddr+4` 就是下一条指令的运行时起始地址, 所以我们需要算 `ADDR(r.symbol) - (refaddr + 4)`, 那么 `addend=-4`.

- 静态链接库 (.a):
    - 创建: `ar rs name.a part1.o part2.o ...`. 查看内容: `ar -t name.a`
    - 链接: 每当链接一个静态库, 尝试在库中寻找所有 **当前** 未成功解析的符号, 如果找到, 用对应定义来解析.
    - 因此需要合理安排链接顺序.
        - 出题的老师/助教似乎从来不考虑交叉循环引用的问题, 例如 `X1->Y1->X2->Y2->...`, 真的需要 `-lx -ly -lx -ly ...`.

- 共享库 / 动态链接库 (.so):
    - 创建: 编译 `.o` 时 **必须** 启用 `-fPIC/-fpic` 以生成位置无关代码 (符合直观理解). 而后 `gcc -shared -o name.so part1.o part2.o ...`.
    - 加载时链接: "仿佛是一个 `.o` 文件, 该干嘛干嘛", 只是在执行时由 dynamic linker 在内存中完成最后链接.
    - 运行时链接 (指定 `-rdynamic` 并编译时链接 `-ldl`):
        - `handle = dlopen("./name.so", RTLD_LAZY)` 加载库.
        - `func = dlsym(handle, "func")` 解析库所需内容; `func` 得到一个函数指针, 可以正常调用.
        - `dlclose(handle)` 关闭库.
    - 需要 **.interp section** 节: 指定动态连接器 (例如 `ld-linux.so`).
    - 需要 **.dynamic section** 节: 指定所需的动态库.
    - 可用 `ldd` 命令查看依赖的共享库. ~~我去, 这玩意儿真的在课件上有? 不把课件背下来你完蛋了!~~
    - 但是代价是什么呢? 需要进入额外的 PLT/GOT 来进行函数调用/访问全局变量 (以实现位置无关).

- 库打桩 (interposition)
    - 编译时打桩: 指定 `-I.` 让编译器优先在当前目录搜索 `include` 文件.
    - 链接时打桩: 指定 `-Wl,--wrap,func` 以指定 linker 参数 `--wrap func`, linker 将 `func` 解析为 `__wrap_func`, 将 `__real_func` 解析为 `func`. 所以需要声明 `__real_func`, 需要定义 `__wrap_func`.
    - 运行时打桩: 自行在共享库中实现同名函数, 通过 `dlopen` 调用库函数.

Exception

On this lonely day / 为逆光的诀别干杯

- An **exception** is a **transfer of control** to the OS kernel in response to some **event**. 很深刻.

- 异常分类:
  - 异步异常:
    - **中断 (interrupts)**:
      - 处理后, 控制交由 **下一条指令** 继续执行 (因为原本的控制流其实没有任何问题, 该干嘛干嘛).
      - 例子: 计时器中断, I/O 中断 (`Ctrl-C`, 网络包抵达, 磁盘数据抵达, ...), ...
    - **信号 (signals)**: **在用户态处理**.
  - 同步异常:
    - **陷阱 (traps)**: 主动地使用 syscall 等陷入 kernel.
      - 处理后, 控制交由 **下一条指令** 继续执行.
      - 例子: syscall, **断点陷阱**, 特殊指令.
    - **故障 (faults)**: 无意触发, **可能** 恢复. **若恢复**, 控制交由 **触发异常的指令重新执行**.
      - 例子: 缺页故障 (恢复之后肯定得重新读内存, *联系虚拟内存一节*), 一般保护性故障 (喜闻乐见的 "段错误", **不可恢复**), 除零异常 (**不可恢复**. 你想恢复之后再除一次零吗?)
      - More: 不可恢复不意味着会被立马终止, 比如我们可以为 `SIGSEGV` 注册 handler, 但如果这个 handler 会返回 (而非终止进程), 控制就真的会交由引发故障的指令, 导致 `SIGSEGV` 再放送. `Ref. intro_segv-Og`
    - **终止 (aborts)**: 无意触发, 没救了.
      - 例子: 非法指令, 奇偶校验错误, 机器检查.
      - More: 其实能被捕获, 但也很危险.

- **Syscall**: 用 `%rax` 传入 syscall number, syscall 也用 `%rax` 返回值.
  - 其他寄存器 (作为 syscall 参数): `%rdi %rsi %rdx %r10 %r8 %r9`. ~~猎奇区.~~

- **进程**:
  - A process is an **instance** of a running program.
  - 内核提供抽象:
    - 独占 CPU 的使用: 来自 **上下文切换**.
    - 独占内存的使用: 来自 **虚拟内存**.
  - 称两个进程 **并发** (concurrently) 运行, 当且仅当它们在 **现实世界** 的运行时间段有交集; 否则称为 **顺序** (sequentially) 运行.
  - **上下文切换**:
    - Processes are managed by a shared chunk of memory resident OS code called the **kernel**. The kernel is **not a separate process**, but rather runs as part of some existing process.
  - **子进程**:
    - 孤儿会挂在 `init` (现在叫做 `systemd`, `pid=1`).
    - Fun fact: 除了 0 号进程 (**空闲进程 / idle task**, 幕后大爹) 外, **一切进程都有父进程**, 例如 `systemd` (台前大爹) 的父进程就是空闲进程.

        - More: 空闲进程字面意思就是空转 CPU, 它保证 kernel 的进程调度有事可干, 并帮助执行等待, 暂停和电源管理等.

- **信号**:
    - kernel 在即将交付控制于某进程用户态时:
        - 计算 `pnb = pending & ~blocked`.
        - 如果 `pnb == 0`, 交付控制. 否则取 **最低位对应信号**, 强迫次进程接收.
        - 用户态的 handler 在返回时会自动触发 `sigreturn`, 它恢复中断时的上下文, 清理内核状态 (例如清除为 handler 设置的 mask), 重新检查信号 (回到第一步).
    - 异步信号...
        - **安全**: `_exit`, `write`, `wait`, `waitpid`, `sleep`, `kill`, ...
        - **不安全**: `printf`, `malloc`, `sprintf`, `exit`, ...

- 相关函数 ~~这也太贴心了~~:
    - `int fork(void)`: "Called once but returns twice." 但应该很熟悉了.
    - `void unix_error(char *msg)` (`csapp.h`).
    - `pid_t getpid(void)`: 返回当前进程 PID.
    - `pid_t getppid(void)`: 返回父进程 PID.
    - `void exit(int status)`: 它是 `[[noreturn]]` 的 (可以作为 "调用系统函数必须检查返回值" 的 **反例**). **不是异步信号安全的!**
    - `int wait(int *child_status)`: 挂起当前进程直到存在子进程终止, 此时回收对应子进程并在 `child_status` 非空时设置子进程退出原因.
        - 状态检查 macros: `WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG, WIFCONTINUED`. 注意必须 **先问是不是 (IF), 再问为什么**.
    - `pid_t waitpid(pid_t pid, int *status, int options)`: 挂起当前进程并等待特定的进程结束.
        - `options` macros:
            - `WNOHANG`: 非阻塞模式, 子进程没结束也立即返回, 此时返回值为 0. ~~空闲进程: 干嘛?~~
            - `WUNTRACED`: 子进程停止时也报告 (配合 `WIFSTOPPED(status)` 检查).
            - `WCONTINUED`: 子进程收到信号恢复执行时也报告 (配合 `WIFCONTINUED(status)` 检查).
    - `int execve(char *filename, char *argv[], char *envp[])`:
        - 在 **当前进程** 加载并运行 `filename`, 可以是可执行文件 **或者** 用 `#!` 指定解释器的脚本文件.
        - **覆写** 代码, 数据, 栈. **保持** PID, 打开文件表, 信号上下文.
        - 除非发生错误, 否则不返回.
        - `envp[]` 是形如 `"name=value"` 的字符串数组, `argv[]` 和 `envp[]` 必须以 `NULL` 作为最后一项. 配合 `getenv`, `putenv`, `printenv` 使用.
            - More: 新进程创建时, `envp[]` 和 `argv[]` 的字符串会先后压入栈底, 然后是二者的指针列表.

- `pid_t getpgrp(void)`: 返回当前进程所属组.
- `int setpgid(pid_t pid, pid_t pgid)`: 设置当前进程所属组.
- `sigprocmask`, `sigemptyset`, `sigfillset`, `sigadddset`, `sigdelset`.
- `int setjmp(jmp_buf j)`, `void longjmp(jmp_buf j, int i)`.
  - **只能 longjmp 到栈上仍然存在的 (called but not returned) 上下文中**. (我总不能帮你编一个新的栈空间吧?)
  - 你猜猜为什么 `setjmp` 只需要传 `jmp_buf j` 而非 `jmp_buf *j`?
    - `typedef struct __jmp_buf_tag jmp_buf[1];`
    - ~~666 数组来了.~~

System-level I/O

我该向谁讨回我的锐利 / 向爱我的人们演示站立

(这里略过了一些我很熟悉以至于不知道记啥的 Linux 文件基础知识.)

- SysIO 调用:
  - 文件描述符. 默认打开 `0: stdin | 1: stdout | 2: stderr`.
  - **开**: `fd = open(filename, options)`.
  - **关**: `close(fd)`. 也要检查返回值.
  - **读**: `nbytes = read(fd, buf, sizeof(buf))`.
    - `ssize_t nbytes` 是 **有符号** 整数 (不然你咋判断它 `< 0`).
    - `nbytes < sizeof(buf)` 是可能的, 并非错误.
      - 读到 `EOF`, 读终端时读到换行, 读套接字.
  - **写**: `nbytes = write(fd, buf, sizeof(buf))`. 要点同上.
  - 读写 syscall $> 10000\text{cyc}$, 所以我们喜欢 buffered I/O.
    - C 标准输出在遇到 `\n` 时会刷缓冲区.
  - **注意**, `char *buf` **并没有被视为 C-style string**, 不会写 `\0`. 所以你甚至可以用 `&c` 作为大小为 1B 的 buffer.

- **文件描述符表 / 打开文件表 / v-node 表**
  - 文件描述符表: 每个进程一套.
  - 打开文件表: **所有进程共享**, 每个 **打开实例** 占一个表项, 描述 **操作这份文件的性质**.
    - 偏移量 (当前读写位置), 访问模式, 状态标志, 引用计数, **指向 v-node 的指针**, I/O 相关信息.
    - `fork/dup/...` 时增加引用计数, `close` 减少引用计数, 计数为 0 时才执行 "关闭文件" 的逻辑.
      - 说白了, `fd` 就该是一个指向打开文件表表项的智能指针, `fork/dup/close` 等函数提供了对引用计数逻辑 "不那么封装" 的封装. 只要你没有干 `int fd2 = fd` 这种事, 一个 `fd` 用完就 `close` 就是合理的做法.
      - 注意 `dup2` 会帮你关旧文件.

- v-node 表: 所有进程共享, **每个文件至多在内存占一个表项**, 所以 **只描述文件本身的性质**.
  - 类型, 大小, 所有者, 权限, 时间戳, ...

- 相关函数:
  - `open`, `close`, `read`, `write`.
    - `O_READONLY`: 只读.
    - `O_WRITEONLY`: 只写.
    - `O_RDWR`: 可读可写.
    - `O_CREAT`: 不存在时创建.
    - `O_TRUNC`: 打开时清空文件内容.
    - `O_APPEND`: 追加模式, **每次写入时** 写在文件末尾.
    - ...
  - `int stat(char *filename, struct stat *s)`: 获取文件元数据.
  - `int dup(int oldfd)`: 省流就是智能指针的 copy.
  - `int dup2(int oldfd, int newfd)`: 省流就是智能指针的赋值构造, 你得先析构原来的东西 (`close(newfd)`).
    - 注意参数顺序不是 `(dst, src)`. 你可以记住 `dup2(X, Y)` 读作 "**dup** X **2** Y".
  - `rio` 系列函数 (`csapp.h`).
  - `DIR *opendir(char *dir_name)`: 打开目录文件.
  - `struct dirent *readdir(DIR *directory)`: 读取目录中的文件信息 (one by one).

Virtual Memory

月亮啊月亮 / 你不在橱窗 / 月亮啊月亮 / 你挂在天上

- 名词解释:
  - **CO**: (DRAM 主存 cache 的) 缓存偏移, **PA** 的最低 b 位 (回忆 Cache 一节的 $S = 2^s$, $B = 2^b$).
  - **CI**: 缓存索引. `CI:CO == VPO == PPO` (对 Core i7 设计成立), **PA** 取走 b 位后的最低 s 位.
  - **CT**: 缓存标记. **PA** 取走 $b + s$ 后剩下的东西.
  - **VP**: 虚拟页, 虚拟内存上的一块连续存储单元, 可能唯一对应一个物理页 (cached) 也可能没有 (访问时产生缺页故障, 内核负责把对应页从磁盘放入内存). $P = 2^p$ 表示页大小.
  - **VA**: 虚拟地址, 在确定进程上下文 (`CR3` 寄存器) 时能够被 MMU 解析. $N = 2^n$ 表示虚拟地址空间. **分为**:
    - **VPO**: 虚拟页偏移, **等于物理页偏移**, 可以在虚拟页向物理页解析的同时发送到 cache 作为 **CI:CO**.
    - **VPN**: 虚拟页编号, 由 MMU 负责翻译为物理页编号. **分为**:
      - **TLBI**: **TLB** 的缓存索引, $T = 2^t$ 个组.
      - **TLBT**: **TLB** 的缓存 tag.
      - **没有** 缓存偏移, 因为命中时一定取出一整个占据若干字节的 **PPN**.
  - **PA**: 物理地址, 指向 DRAM 主存上的真实位置. $M = 2^m$ 表示物理地址空间.

        - **PPO**: 见 VPO.
        - **PPN**: 物理页编号, 也是 cache 的 **CT**.
    - **MMU**: 内存管理单元. 完成 **VPN** 到 **PPN** 的转换逻辑.
    - **PTBR**: 页表基址寄存器, Linux 中叫 `CR3`, 是进程上下文的一部分, 用于为每个进程指明页表头, 进而隔离进程内存.
    - **PT & PTE & PTEA**: 页表, 页表项和页表项地址. 每个页表项记录了 **valid bit** (指向页是否在内存中) 和指向的 **PPN**. 页表表头由 **PTBR** (一级页表) 或者上一级页表 (其他级页表) 指定. **PTEA** 时 **MMU** 解析 **VPN** 时向内存 (或 cache) 发送的指向目标 **PTE** 的 **PA**.
    - **TLB**: Translation Lookaside Buffer,《转译后备缓冲器》或者像人话一点的 "页表缓存". **是 MMU 内部的小 cache**, 用来缓存 **VPN** 到 **PPN** 的结果, 可见它 **对进程切换敏感**.

- 一场酣畅淋漓的寻址!
    - 以 **Intel Core i7** 参数为准:
        - $N = 2^{9\times4+12} = 2^{48}$, $M = 2^{40+12} = 2^{52}$.
        - L1 d-TLB $s = 4$, $E = 4$; L1 i-TLB $s = 5$, $E = 4$;
        - L2 unified-TLB $s = 8$, $E = 4$.
        - L1-cache $b = 6$, $s = 6$.
        - 四级页表, 支持大小为 $2^{12} = 4\text{KB}$, $2^{12+9} = 2\text{MB}$, $2^{12+9\times2} = 1\text{GB}$ 的页. **教材和课件参数是 32 位的, 有误, 以这里为准**.
            - 不支持 512GB 超大杯: 一级页表没有 `PS` 位.
            - **注意**, 我查资料发现 "所有支持x86-64架构的Intel Core i7处理器（自Nehalem架构起）都支持 1GB 大页", 但课件上声称只有三级页表有 `PS`, 也就是只支持小杯和中杯, 不支持大杯. **这个需要你自行衡量.**
    - **小端序! 小端序! 小端序! 小端序! 小端序!**
    - CPU 发起对 `VA[47..0]` 的读, 暂不考虑写, 暂不考虑缺页, 暂不考虑权限, 不考虑大页.
    - `VPN, VPO = VA[47..12], VA[11..0]`.
    - `PPO = VPO`, `PPO` 发到内存 cache 中 (只考虑一级缓存):
        - `CI, CO = PPO[11..b], PPO[(b-1)..0]`.
        - 准备对 `CI` 组的命中检查, 等待 `CT`...
    - **同时**, `TLBT, TLBI = VPN[35..t], VPN[(t-1)..0]`.
    - 在 **TLB** 中寻找 `(TLBT, TLBI)`:
        - 若命中:
            - `PPN = cached_ppn`.
        - 否则:
            - `VPN1, VPN2, VPN3, VPN4 = VPN[35..27], VPN[26..18], VPN[17..9], VPN[8..0]`.
            - `PPN1 = PTBR[VPN1].ppn`, `PPN2 = PPN1[VPN2].ppn`, `PPN3 = PPN2[VPN3].ppn`, `PPN = PPN3[VPN4].ppn`.
    - `CT = PPN`, 发到内存 cache.
        - 若命中, 返回对应数据;

        - 否则, 向下一级缓存或 DRAM 发送 `PPN:PPO = CT:CI:CO`.

- 上下文切换: 刷新 TLB / 在 TLBE 中附加上进程 ID 信息.

- 冷知识: 四级页表分别叫做 page global/upper/middle directory 和 page table.

- PTE 信息 (除 40 位 PPN 外):
    - `P=0/1`: 子页 (表) 是否在内存中. `P=0` 时其他标记无意义 (此时 kernel 可以用剩余区域记录其他信息).
    - `R/W`: 只读还是可写. (标记可达的所有页的权限.)
    - `U/S`: 用户态可访问还是内核态可访问. (标记可达的所有页的权限.)
    - `WT=0/1`: 是写回还是写穿透. 标记指向的页 (表) 的缓存策略.
    - `A`: 引用位. MMU 读写时都标记, 由软件清除.
    - `PS` (对二, 三级页表): 指向的是否是一整个大页; `D` (对四级页表): dirty bit.
    - `XD`: 是否可以在页上取指令来执行. (标记可达的所有页的权限).

- 每个进程持有一个 `task_struct`, 成员 `mm` 指向一个 `mm_struct`, 它包含:
    - `pgd`: 第一级页表 (名字缩写) 的头地址;
    - `mmap`: 虚拟内存区域链表头, 指向一个 `vm_area_struct`, 它包含:
        - `vm_end, vm_start`: 框定一块内存区域;
        - `vm_prot, vm_flags`: 标定读写权限, 是否私有;
        - ...
        - `vm_next`: 指向链表下一个元素.
    - **COW** (copy on right): PTE 上记为只读, `vm_flags` 记为 private & copy-on-write, 触发故障时 handler 负责复制出一块可读写的新内存.
    - **demand-zero page** (例如 `.bss`) 由一种 **匿名文件** 提供 (第一次写故障时, 将全 0 的匿名文件映射向对应内存区域).

- `void *mmap(void *start, int len, int prot, int flags, int fd, int offset)`:
    - "将 `fd` 文件偏移 `offset` 字节开始 `len` 个字节的内容映射到 `start` (0 或指定的地址) 开始的内存区域, 并标记对应保护和标志, 返回真正映射到的起始地址."
    - 用 `MAP_SHARED` 标志让多个进程共享同一份文件 (进而共享一段内存), 对内存的修改会同步到文件.
    - 用 `PROT_WRITE` 实现进程内存-磁盘文件级别的 write-back (例如数据库等).

Dynamic Memory Allocation

谁将重力悬空 / 坠入一场蔚蓝的梦

- 名词解释:

- **payload**: 真正用于存放程序数据的内存大小.
- **overhead**: 当前堆总大小 / 累计 payload 的峰值 - 1.
- **内部碎片**: 一块事实上被分配区域中不用来存放程序数据的区域 (堆数据结构, 对齐, 主动分配大块等).
- **外部碎片**: 没被分配, 但不够连续以至于无法满足分配需求的区域.

- 堆数据结构与分配策略:
  - **隐式链表**: 块头存块大小, 是否分配; 块脚和块头相同 (用于合并).
    - 优化: 已分配块不保留块脚; 在块头记录前一块是否被分配.
  - **显式链表**: 未分配块用预留的 payload 区域存双向链表.
  - **分隔空闲列表**: 按块大小分组, 每组开显式链表.
  - **有序空闲块**.
  - **初次适配**, **下次适配**, **最佳适配**.

- 冷知识:
  - `valgrind`.
  - `glibc` 提供了检查: `setenv MALLOC_CHECK_ 3`.

- 垃圾回收
  - Mark & Sweep: 暴力得一. 先把每个块头打上不可达标记, 然后从所有疑似指针的位置出发不断标记可达的块, 最后回收所有仍然不可达的块.

Network Basis / Programming

雪来临时 / 灯火煌煌 / 喧哗擦肩而过追问着何时归家

(网络基础部分真的看麻了... 没怎么记.)

- 分层互联网协议栈 (Layered Internet Protocol Stack):
  - **应用层** "传什么数据? 拿来干什么?"
    - e.g. HTTP, IMAP, SMTP, DNS.
  - **传输层** "从哪里到哪里?"
    - e.g. TCP, UDP.
  - **网络路由层** "途经那些地方?"
    - e.g. IP, routing protocol.
  - **链路访问层** "有线还是无线? 下一站是哪里? 链路可访问吗?"
    - e.g. Ethernet, 802.11 (WiFi), PPP.
  - **物理层** "具体怎么传每个 bit? 什么调制格式?"
  - 数据层层打包层层解包.
    $[M] \to [H_t \mid M] \to [H_n \mid [H_t \mid M]] \to [H_l \mid [H_n \mid [H_t \mid M]]] \to \cdots \to M.$

- **TCP/IP 协议族**
  - **IP** (Internet Protocol): 提供 **主机到主机** 的基本命名规则 (basic naming scheme) 和不可靠的数据包传输能力.
  - **UDP** (Unreliable Control Protocol): 使用 IP 提供 **进程到进程** 的 **不可靠** 数据包传输.
  - **TCP** (Transmission Control Protocol): 使用 IP 提供 **进程到进程** 的 **可靠** 的字节流 **连接**.
    - 全双工: 双向数据可以同时流动.
    - 可靠: 发的东西最终总能被对方正确地收到.
  - 套接字提供用户态 (client) 到内核态 (TCP/IP) 的接口 (syscall).

- Internet 相关
  - **IP 地址**, **域名**, **连接**.
  - **IPv4 (32 bit)**, **IPv6 (128 bit)**.
  - **子网**: e.g. `w.x.y.z/n`, `n` 表示子网掩码位数, 即高 n 个 bit 是子网地址, 剩下 $32-n$ 个 bit 是主机地址. *(课件中部分表述有误.)*
  - *传统* 分类:
    - A 类: `(0:w').x.y.z/8`.
    - B 类: `(10:w').x.y.z/16`.
    - C 类: `(110:w').x.y.z/24`.
    - D 类: `(1110:w').x.y.z` 组播地址.
    - E 类: `(1111:w').x.y.z` 保留/实验地址.
    - **特殊**: 私网地址 `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`.
  - `struct in_addr { uint32_t s_addr; };` **总是大端序存储**.
  - `.com` 等叫做一级域名 (根域名不算).
  - **DNS**: `nslookup <域名>` 查询. 一对一, 一对多, 多对一, 无对应, 都有可能.

- **Socket**
  - `socket_addr = (IP_addr, port)`.
  - port 是 16 位整数, 分为临时端口 (ephemeral: 朝生暮死的 ~~居然认识~~) 和知名端口.
  - **知名端口** 与特定的服务对应:
    - **echo**: 7.
    - **ftp**: 21.
    - **ssh**: 22.
    - **smtp**: 25 (for email).
    - **http**: 80.
    - **https**: 443.
    - 映射保存在 `/etc/services`.
  - socket pair 唯一标识了一个连接.

- **URL** (Universal Resource Locator):
  - URL 是一种 **URI** (Universal Resource Identifier).
  - 顾名思义, URL 告诉你资源如何访问 (一定能以此访问), URI 告诉你资源是什么 (但不一定可以以此访问).

- **CGI** (Common Gateway Interface):
  - e.g. `/cgi-bin/adder?123&456`.
  - 通过环境变量 `QUERY_STRING` 传递给 CGI 程序.

- 相关函数:
  - `int getaddrinfo(const char *host, const char *service, const struct addrinfo *hints, struct addrinfo **result)`
    - 查询 `host(域名 / IP 地址 / NULL):service(服务名称 / 端口 / NULL)` 的所有满足 `hints` 过滤条件的 socket 地址信息. (链表项可以直接用于 `socket` 函数连接).
    - 释放: `void freeaddrinfo(struct addrinfo *result)`.
    - 错误解释: `const char *gai_strerror(int errcode)`.
    - `NULL` 类似通配符, 查询到的 `result` 也可以理解作通配地址.
  - `int getnameinfo(const SA *sa, socklen_t salen, char *host, size_t hostlen, char *serv, size_t servlen, int flags)`
    - 上一个函数的 "反函数".
  - `int socket(int domain, int type, int protocol)`.
    - e.g. `int clientfd = socket(AF_INET, SOCK_STREAM, 0)`, 建立 IPv4 / TCP socket 描述符.
    - 推荐: 使用 `getaddrinfo` 的结果进行协议无关编程. `socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol)`.
  - `int bind(int sockfd, SA *addr, socklen_t addrlen)` (多用于 server): 建立 socket FD `sockfd` 与 `addr` 端口的联系. (若采用 UDP) 所有抵达该端口的信息可由 `sockfd` 读, 写向 `sockfd` 的数据会广播到所有该端口的链接.
  - `int listen(int sockfd, int backlog)`: 将 `sockfd` 从一个 active socket 转化为一个 listen socket, 告诉内核监听向这个 `sockfd` 的连接请求. `backlog` 可以指定内核开始拒绝 (refuse) 请求前最多允许多少请求等待.
  - `int accept(int listenfd, SA *addr, int *addrlen)`: **阻塞**, 直到一个向 `listenfd` 的连接请求到达. 记录客户端 socket 信息和长度, 并返回 connected descriptor (客户端与服务器的 socket pair 确定了这个 connection).
  - `int connect(int clientfd, SA *addr, int *addrlen)`: 尝试向 `addr` 所指定的服务器建立连接, 若成功, `clientfd` 称为对应的描述符.
    - `clientfd` 一开始是一个客户端的 socket 描述符.
    - 仍推荐使用来自 `getaddrinfo` 得到的 `addr` 和 `addrlen` 信息.
  - `open_clientfd(char *hostname, char *port)` (`csapp.h`) 流程:
    - 默认使用 `SOCK_STREAM` 即 TCP 连接, 调用 `getaddrinfo` 查询 `hostname:port` 上的可用 socket 地址信息.

        - 遍历信息列表, 逐个尝试:
            - 调用 `socket`, 建立与服务器协议对应的 socket 描述符 `clientfd` (失败则放弃次连接).
            - 调用 `connect` 尝试将 `clientfd` 向服务器连接 (成功则跳出).
            - 关闭 `clientfd`.
        - 清空信息列表.
        - 返回成功建立连接的 `clientfd` 或 `-1`.
    - `int open_listenfd(char *port)` (`csapp.h`): 打开监听 `port` 端口的描述符.
        - (藏了不少细枝末节的操作, 这里略过.)

Concurrent Programming

反复确证心愿向天空抛洒花瓣 / 相遇离别皆献上星光一盏

- Joinable & Detached:
    - **Joinable**: 表明线程需要被其他线程回收以释放内存资源.
    - **Detached**: 不能被其他线程回收或杀死.
        - 可用 `pthread_detach(pthread_self())` 把自己 detach 掉.

- 进程图, 轨迹:
    - **进程图** 是离散的执行空间 (那个坐标轴) 和锁框出的禁止区域.
    - **轨迹** 是一个进程实例跑出来的折线图.
    - 发现死锁: 禁止区可能卡住从原点出发的某条折线 (轨迹).

- **信号量**
    - 非负整型全局变量.
    - `P(s) / V(s)`: "尝试拿走一个令牌 / 放回一个令牌". 良好的比喻.
    - **Binary semaphore**, **Mutex**, **Counting semaphore**.

- **第一类读者-写者问题**
    - 基本约束: **写者独占**, **读者并发**.
    - 第一类要求: *希望* 同时有读写者等待时优先唤醒读者.
    - *这对吗?*

```c
int readcnt; /* Initially 0 */
sem_t mutex, w; /* Both initially 1 */

void reader(void)
{
    while (1) {
        P(&mutex);
        readcnt++;
        if (readcnt == 1) /* First in */
```

```
            /*
                问题: 若此时正有写者写, 没有读者. 新读者会在 w 阻塞, 然后又有新写者,
                写者也会在 w 阻塞, 我们没办法保证阻塞在这里的读者先被唤醒.
                Even worse, 如果有一车写者阻塞在 w, 但由于 mutex 被第一个阻塞在 w 的读者持有,
                只能有这一个读者参与 w 锁的竞争. 在 "公平拼手速" 的预设下这个读者根本抢不过写者. (
            */
            P(&w);
        V(&mutex);

        /* Reading happens here */

        P(&mutex);
        readcnt--;
        if (readcnt == 0) /* Last out */
            V(&w);
        V(&mutex);
    }
}

void writer(void)
{
    while (1) {
        P(&w);
        /* Writing here */
        V(&w);
    }
}
```

- **第二类读者-写者问题**
  - 基本约束: 同上.
  - 第二类要求: *希望* 同时有读写者等待时优先唤醒写者.
  - *这对吗?*

```
int readcnt, writecnt; // Initially 0
sem_t rmutex, wmutex, r, w; // Initially 1

void reader(void)
{
    while (1) {
        P(&r);
        P(&rmutex);
        readcnt++;
        if (readcnt == 1) /* First in */
            P(&w);
        V(&rmutex);
        V(&r);

        /* Reading happens here */

        P(&rmutex);
        readcnt--;
```

```c
        if (readcnt == 0) /* Last out */
            V(&w);
        V(&rmutex);
    }
}

void writer(void)
{
    while (1) {
        P(&wmutex);
        writecnt++;
        if (writecnt == 1)
            P(&r);
            /*
                还是有一样的问题. 一个抢 r 的写者和一车抢 r 的读者拼手速.
                教材似乎觉得写者能抢得过?
            */
        V(&wmutex);

        P(&w);
        /* Writing here */
        V(&w);

        P(&wmutex);
        writecnt--;
        if (writecnt == 0);
            V(&r);
        V(&wmutex);
    }
}
```

- **爱来自助教**: 关于读者-写者问题.
  - "对于第二类问题或许可以这样理解: 如果按照第一类的实现, 如果有源源不断的读者, 那么写者在机制上就无法被执行而按照第二类的实现, 就可以通过竞争锁的方式而拿到执行的机会."
  - "如果题目按照一般的说法 "赋予优先级", 然后我发现这种问题, 我可能真不知道咋写?" - **"那就按照赋予读者优先级就是第一类, 赋予写者优先级就是第二类理解就行."**
  - "所以某种程度上我们的目标是 **在任何构造性情景下**, 读者/写者都 **有机会** 竞争到另一方持有的锁, 这样就达成了所谓的读者/写者 **优先**?" - "是的."

- **读者-写者 FIFO 实现**
  - 对完了.

- **线程安全不一定是异步信号安全的, 反之正确.**
  - 所有 Standard C Library 都是线程安全的.
  - 绝大多数 Unix syscall 是线程安全的. 例外:
    - `asctime`, `ctime`, `gethostbyaddr`, `gethostbyname`, `inet_ntoa`, `localtime`, `rand`.

- 相关函数
  - `int pthread_create(pthread_t *tid, NULL, void (*)(void*) thread, void *vargp)`.
  - `pthread_join(pthread_t tid, void **return_val)`.
  - `pthread_self()`.
  - `pthead_exit()`.
  - `exit()`: 退出 **进程**, 无论当前线程是否 detached, 所有线程都随进程终止.
  - `pthread_mutex_init()`.
  - `pthread_mutex_[un]lock()`.